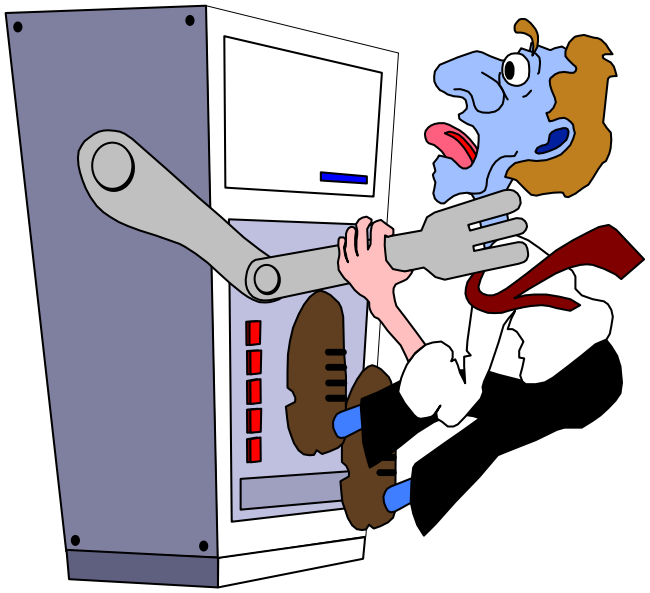
Test

Chapter 11

Why Test?

- Productivity losses, missed opportunities
- User frustration, complaints, and increased Help Desk calls
- System outages and maintenance
- Lost or corrupted data



Testing - Definitions

- Executing or evaluating a system by manual or automated means to
  - verify that it satisfies specified requirements
  - find differences between expected/actual results
- Demonstrates correspondence between an element and its specification
  - no testing without a specification of intentions
- An attempt to demonstrate that an element does not meet its specification

Some Purposes of Testing

- Demonstrate that the system will accomplish its mission
- Prove that an identified set of functions will operate correctly under given circumstances
- Demonstrate that a system does not meet its specification
- Identify defects or common defects
- Improve confidence that a system will accomplish its mission

Issues With Testing

- Testing can prove that the system does not meet its specification
  - cannot *prove* that system meets its specification
- Testing of all possible paths through the software is impossible
  - combinatorial explosion
- External conditions affecting the software execution are practically unlimited
- Thus: if you have to be absolutely certain that software is correct, testing can't do that!

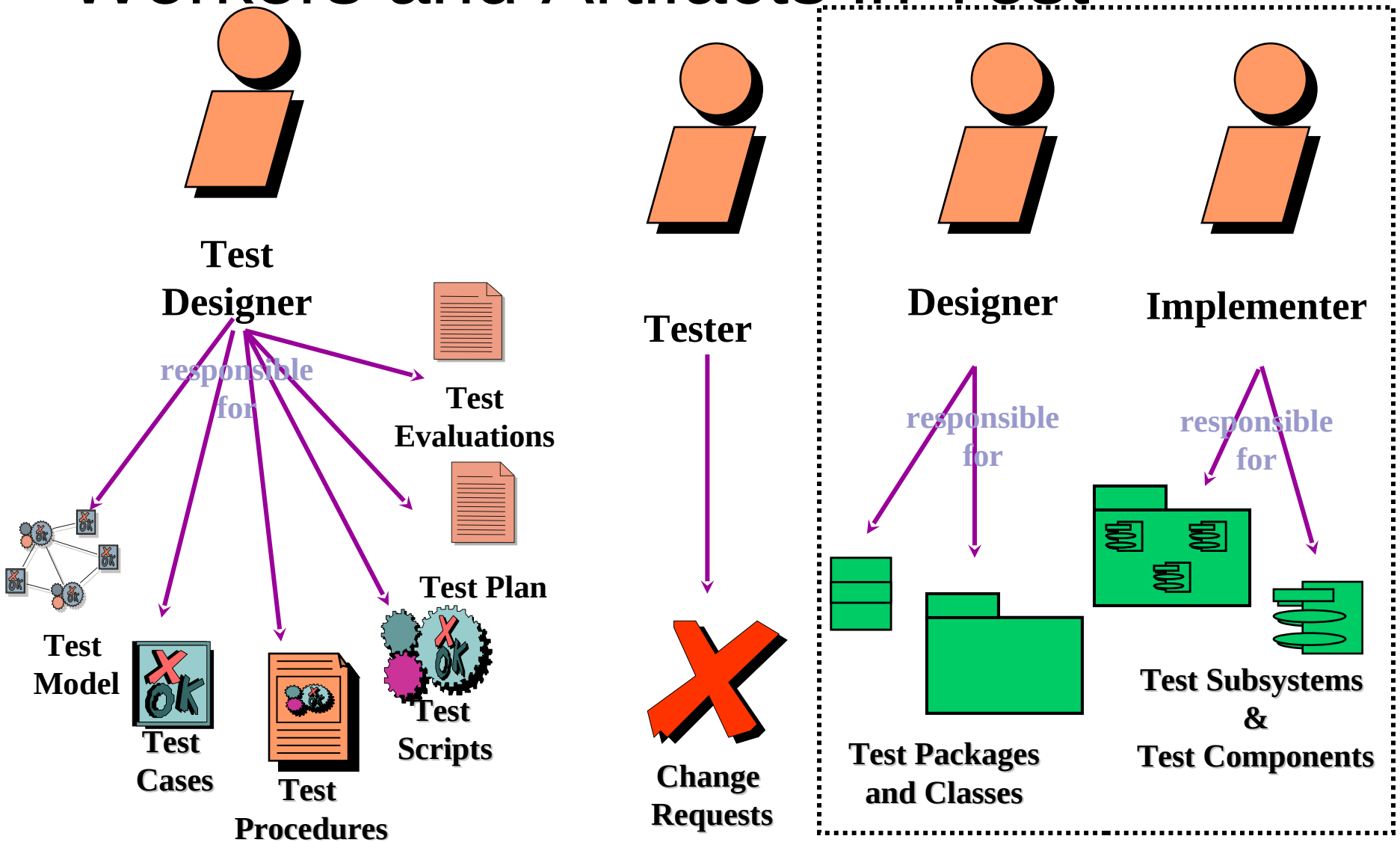
Tests As Procedures

- Tests are formal procedures
- Must be repeatable and yield the same results
- Must be designed, documented, reviewed and debugged.
- Errors could occur in a test just as easily as in the system being tested

Testability

- How easy is it to test:
  - requirements, designs or code
- Major design decision:
  - how can this element be tested thoroughly
- Should not allow any non-verifiable requirements
  - some may have to be verified by other means than testing
- Testability doesn't just happen
  - have to consciously build it into a system

Workers and Artifacts in Test



Test Case

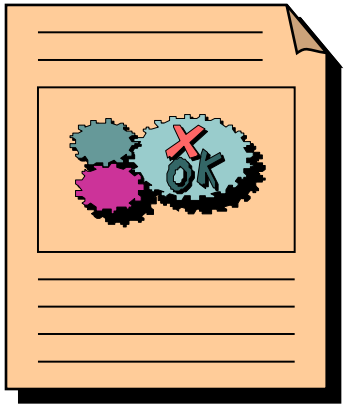
A written document developed for a specific test requirement that consists of:

- Test inputs
- Execution conditions
- Expected results (verification point)

Test Cases

Test Procedure

A written document that consists of a set of detailed instructions for the setup, execution, and evaluation of a test case (or set of test cases)

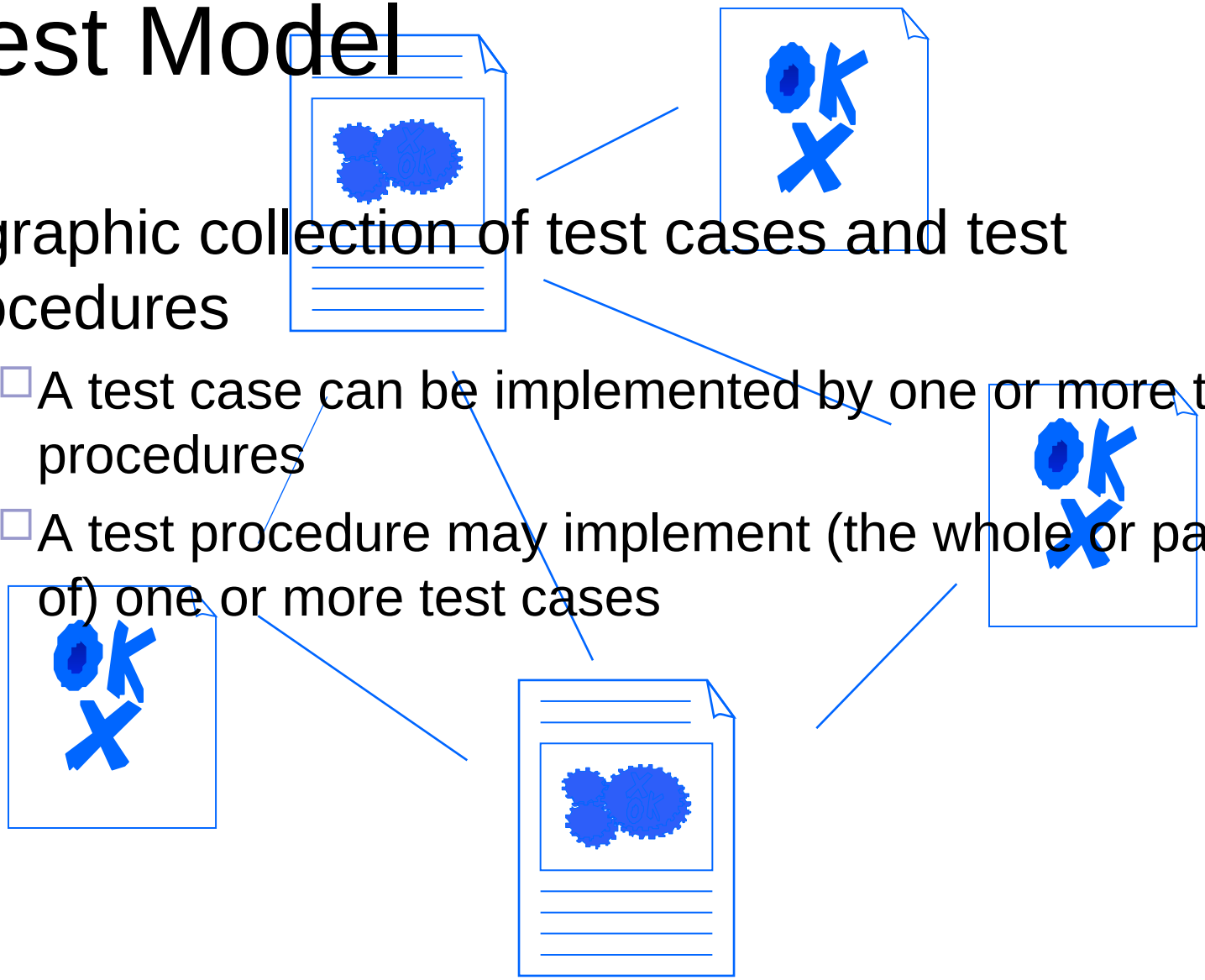


Test Procedures

Test Model

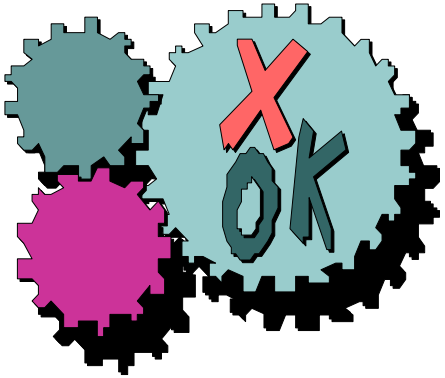
A graphic collection of test cases and test procedures

- A test case can be implemented by one or more test procedures
- A test procedure may implement (the whole or parts of) one or more test cases



Test Scripts

- Computer-readable sets of instructions that automate the execution of a test procedure (or portion of a test procedure)
- Not needed for manual testing

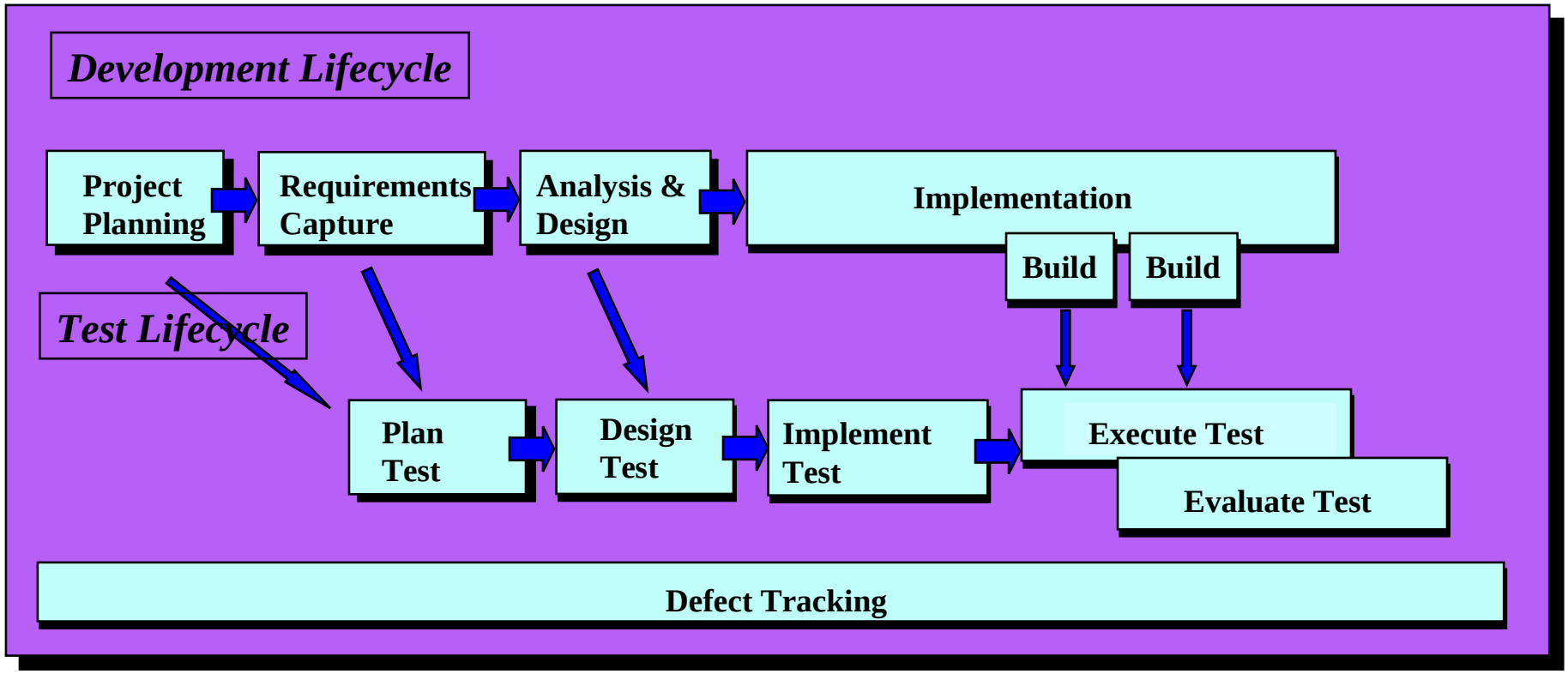


Test Scripts

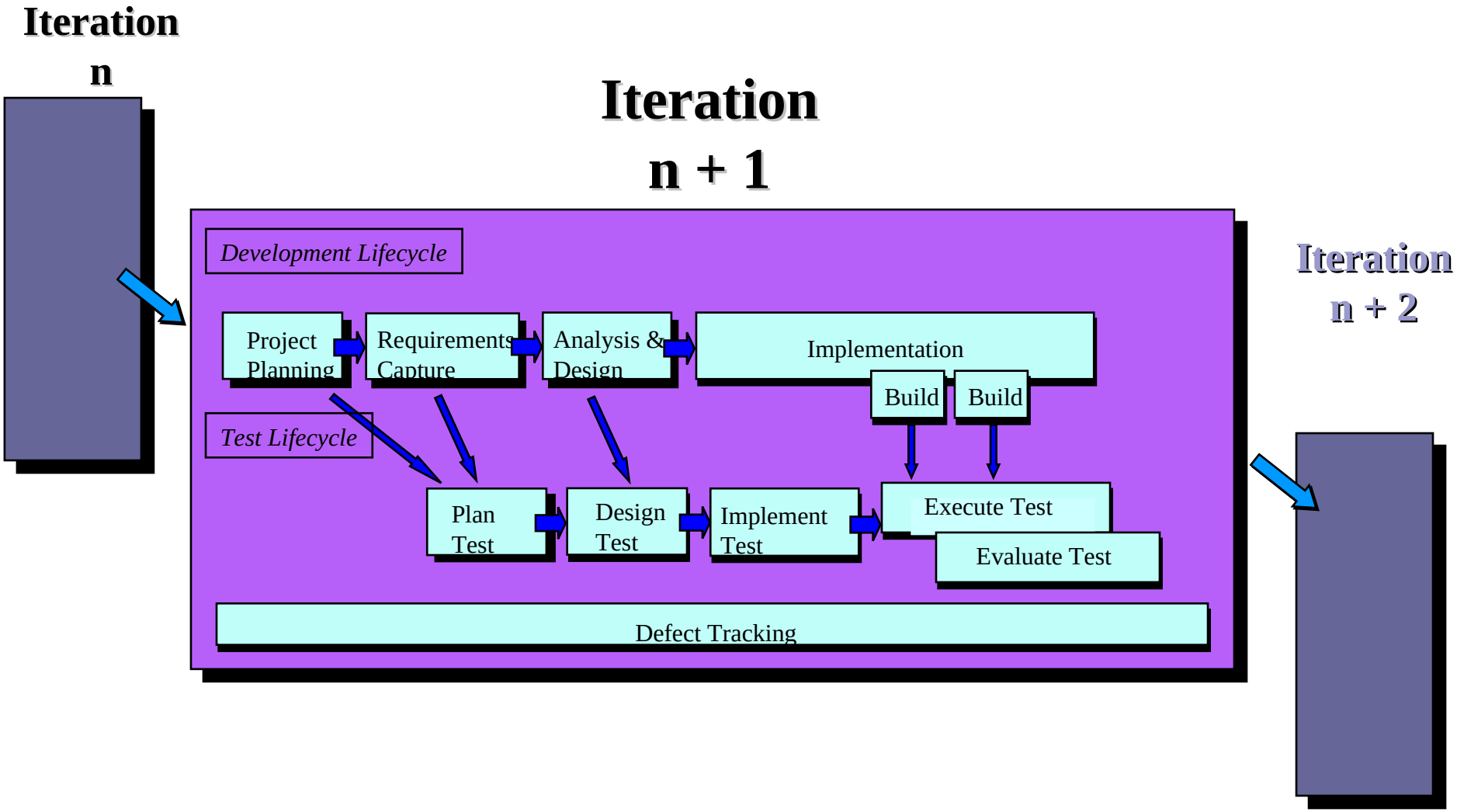
Verification Methods

- Approaches used to verify a verification point
- Needed for manual and automated testing
- Manual examples:
  - Calculate
  - Look
- Automated examples:
  - Object properties
  - Window existence
  - Object data

The Testing Lifecycle

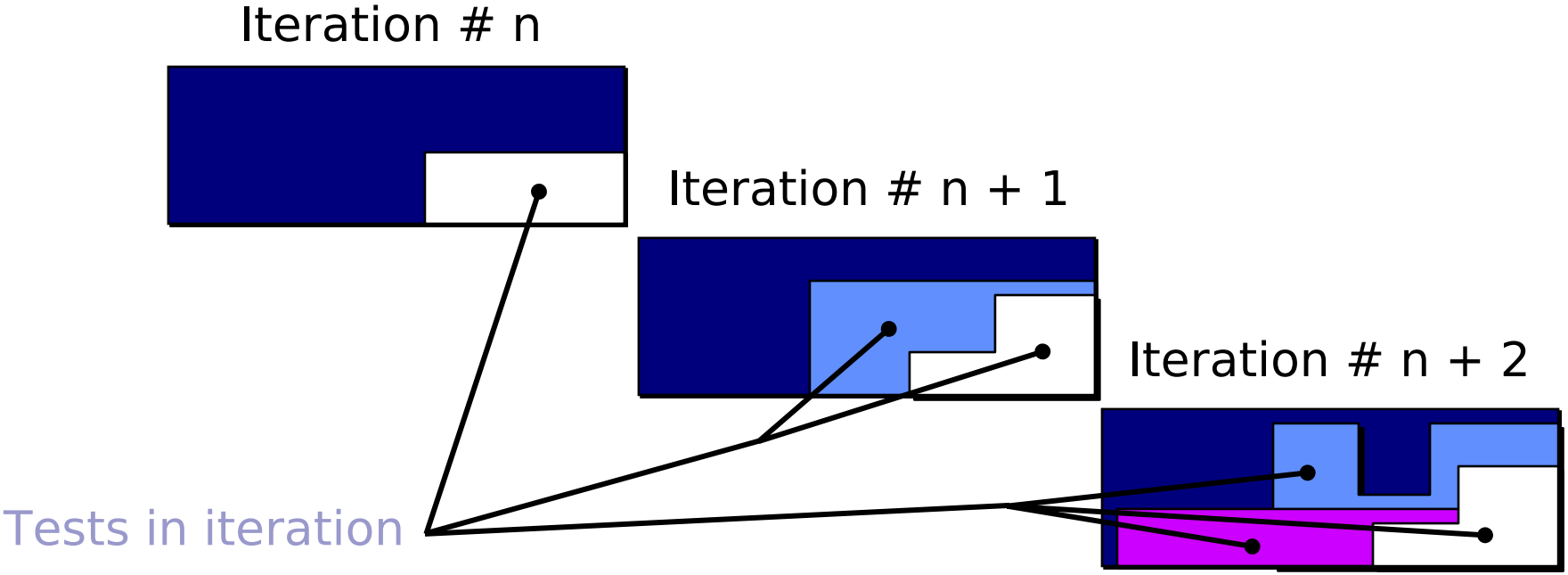


Iterations

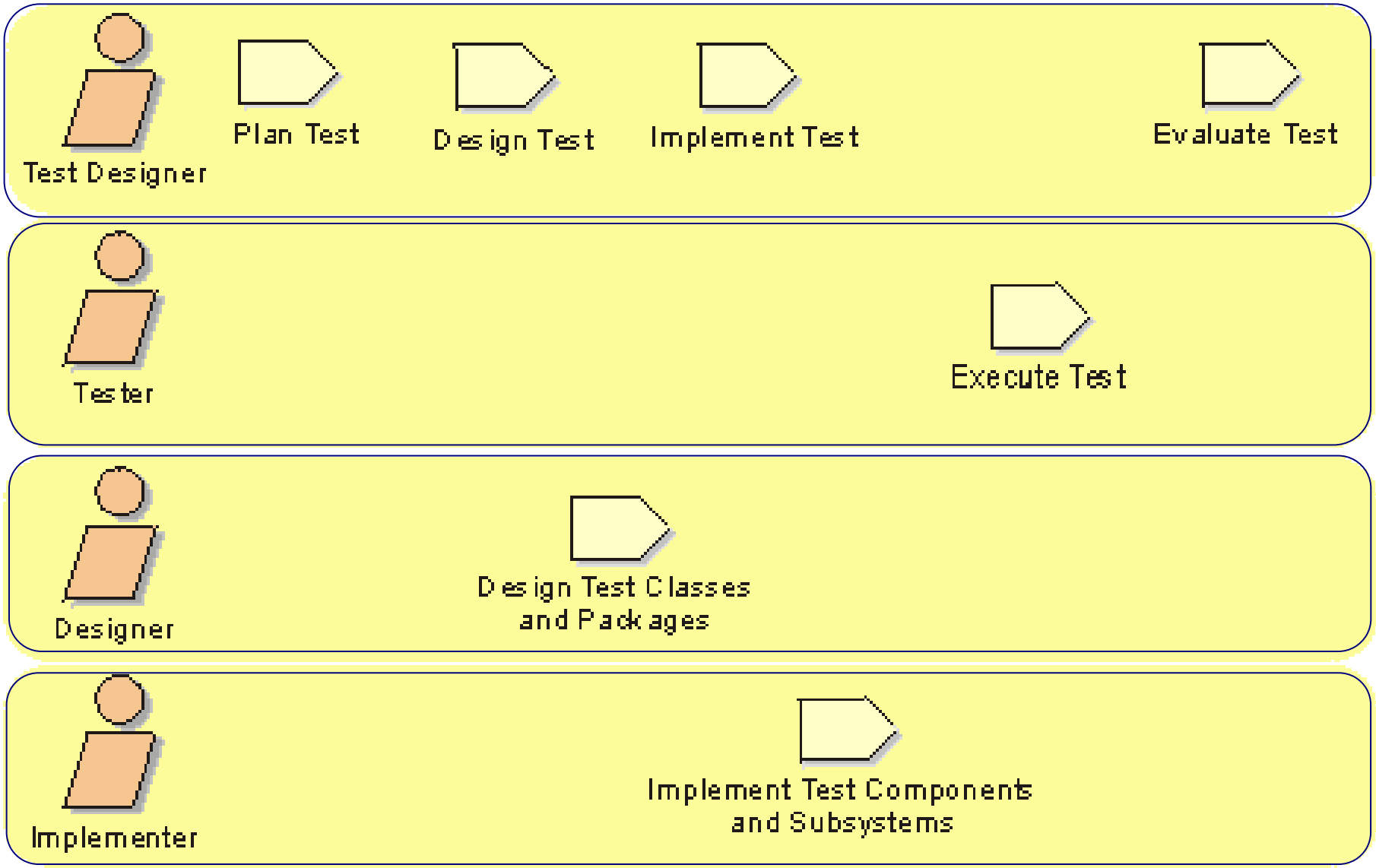

Iteration n
Iteration n + 1
Development Lifecycle
Project Planning
Requirements Capture
Analysis & Design
Implementation
Build
Build
Test Lifecycle
Plan Test
Design Test
Implement Test
Execute Test
Evaluate Test
Defect Tracking
Iteration n + 2

Iterations

Tests evolve in iterations



Test Activity Overview



Plan Test

- Define the test strategy
- Identify the test resources
- Identify the requirements for test
- Generate the test plan

Design Test

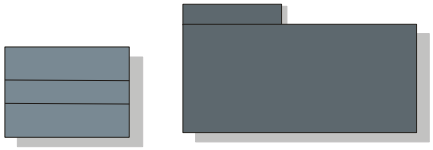
- Test cases
- Test procedures
- Verification methods
- Test scripts

Implement Test

- Produce test procedures by:
  - Creating
  - Acquiring
- Record test scripts

Design Test Classes and Packages

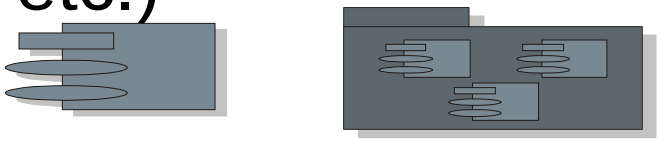
- Identify test-specific classes and packages (stubs, drivers, etc.)
- Design interface to automated test tool
- Design test procedure behavior



Test Packages and Test Classes

Implement Test Components/Subsystems

Implement test-specific classes and packages (stubs, drivers, etc.)



Test Subsystems and Test Components

Execute Test

Execute test cases and procedures

- Play back test scripts
- Test manually

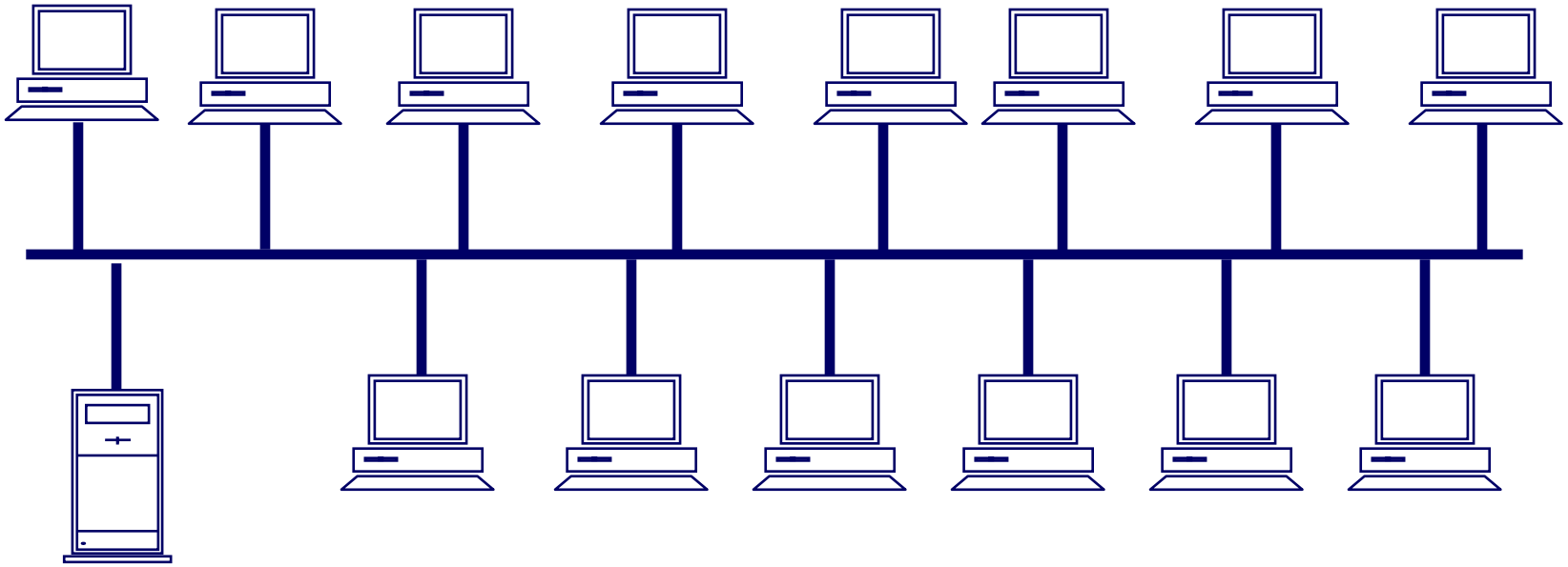
Evaluate Test

- Evaluate results from test execution
- Assess completion criteria

Kinds of Testing

- Testing to verify different requirements:
  - Business function/rules
  - User interface
  - Input validation/syntax testing
  - Performance
  - Configuration/installation
  - Volume
  - Stress
  - Security
  - Recovery
  - Documentation

The Target of the Tests



Business Functions and Rules Test

- Verifies that the software provides the expected services
- Includes:
  - Valid or positive conditions
  - Invalid or negative conditions
  - Business rules and workflow
- Derived from:
  - Use cases
  - Supplementary specifications
  - Activity diagrams

User Interface Test

- Verifies the look and operation of UI
- Includes:
  - Look and feel
  - Navigation
  - Consistency
  - Compliance to standards
- Derived from:
  - Supplementary specifications
  - UI standards and guidelines

Usability Testing

- Testing the system to find human-factor problems
  - user response time can’t keep up with the software
  - user reflexes are stressed by the software
  - messages are not straight-forward and intuitive
  - user does not know how to recover from an error

Input Validation/ Syntax Testing

- Correct or invalid input forms
  - can handle the specified or designed user command syntax
  - any single key is depressed on the keyboard?
  - garbage strings are input?
- Combinations of inputs may be very large
  - tests must be selected to exercise all features of the syntax, and test all classes of erroneous inputs

Performance Test

- Verifies the response and processing time
- Focuses on response time
- Includes:
  - Varying system components to optimize response time
  - End-to-end and/or specific subsystems
  - Use of a limited number of available application’s functions
- Derived from supplementary specifications

Load Test

- Verifies that the system supports the required number of simultaneous clients
- Focuses on proper function and response time
- Includes:
  - Varying clients/transactions (optimal → worst)
  - End-to-end test
  - Large sampling of application functions, modeled after the different actor's work
- Derived from supplementary specifications

Configuration Test

- Verifies proper functionality in different hardware and software environments
- Focuses on acceptable functionality
- Includes:
  - Selected business function tests (regression test)
  - Hardware and software environment specific tests
- Derived from supplementary specifications

Installation Test

- Verifies installation of application in different hardware and software environments or configurations
- Focuses on installation process
- Includes:
  - Varied installation options (normal, custom)
  - Varied configurations
  - Selected business function tests (regression test)
- Derived from supplementary specifications

Volume Test

- Verifies the support of large amounts of data
- Focuses on DBMS transactions and functions
- Includes:
  - Large amount of data in DB
  - Large amount of data in single transaction
- Derived from supplementary specifications

Stress Test

- Verifies that system functions acceptably when specific conditions are encountered
- Includes:
  - Low memory or disk space (client and servers)
  - System resources
  - Database locks
  - Network bandwidth
- Derived from supplementary specifications
- Can usually be done in a black box fashion

Security Testing

- Tries to demonstrate that unauthorized access to the system is impossible
- Requires testing at all interfaces for normal modes of access
- Also requires testing to determine if access can be achieved through extraordinary means
  - can the system be probed by external software or is its memory secure?
  - Is there physical security?

Recovery Testing

- Tests the ability of the system to detect and recover from errors
  - applicable to 'fault tolerant' software
- Difficult to test in a non-instrumented way, so usually is done in a white box fashion
  - e.g. show that in a redundant system, if one piece of software is out of sync with the others, it will shut-down or re-initialize
  - e.g show that after a power failure, the system comes back up correctly

Documentation Test

- Verifies that the documentation conveys the appropriate and correct information
- Focuses on the information provided
- Includes:
  - Online
  - Written
  - Context-sensitive help
- Derived from supplementary specifications

Test Case Selection

- Full testing of all possible inputs or covering all possible paths is impossible
- Want to find ways to limit the number of inputs
  - and thus the number of test cases
- Also want to select the right inputs to maximize the probability of finding errors
- Talked about branch testing and use-case testing earlier
- Next slides look at some additional strategies

Equivalence Partitioning

- Partition the inputs into equivalence classes
  - so a test of a representative value of each class is equivalent to a test of any other value
  - if one test case in an equivalence class detects an error, all other test cases in the same class would be expected to find the same error
- Example: if an input value can have a range from 0 to 999, three equivalence classes stand out:
  - (1) input < 0
  - (2) input from 0 to 999
  - (3) input > 999

Equivalence Partitioning

- Example 2: if an input string must start with a letter then two obvious equivalence classes are:
  - (1) inputs starting with a letter
  - (2) inputs not starting with a letter
- What other reasonable equivalence classes are missing from this?

Boundary Value Analysis

- Test cases that explore boundary conditions have a higher payoff
  - more errors occur at boundary conditions
- Define equivalence classes as before
  - then find the boundaries of each equivalence class and test them
- Boundary cases for earlier example:
  - (1) -1, and maximum negative integer
  - (2) 0, and 999
  - (3) 1000, and maximum integer

Error Guessing

- A largely intuitive and ad hoc process
  - classes of errors are defined and test cases that might isolate errors of these classes are identified
- Test classes can be driven from IEEE Std 1044 or experience
- Start with use-cases
  - identify the classes of errors that could reasonably occur executing that use-case
  - invent test cases to detect those errors

Review

- What are the major activities of the test lifecycle?
- What is the purpose of each?
- Why plan for testing?
- What is testing?
- What are the phases in the testing lifecycle?
- What are the testing stages?
- What are the types of testing?